

Project 2: Dijkstra's Algorithm

Team 7

Members:

MADHAV RAGHU ANANTHARAM

GONG YICHEN

SEAN TEO WEI XIANG

LYE CHEE HUNG IRWIN

Dijkstra's Algorithm Overview

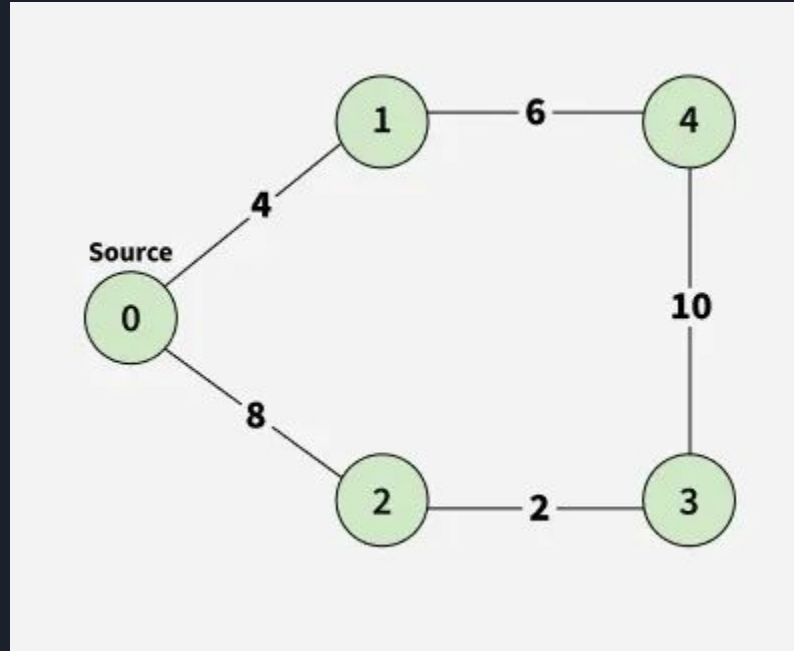


Fig1. Dijkstra's Algorithm shortest path problem



Dijkstra's Algorithm Overview

What it is: A classic algorithm to find the shortest paths from a single source node to all other nodes in a weighted graph.

Key requirement: Edge weights must be **non-negative**.

Core idea (process):

- Start from a source node.
- **Iteratively** select the unvisited node with the **smallest known distance**.
- Update the distances of its neighbors.
- This process repeats until all nodes have been processed.

Challenge: The efficiency of the main operations depends heavily on **the choice of data structures**.



What Affects the Performance?

1. Graph Representation

How do we store the graph and efficiently access a node's neighbors?

- Should we use an **Adjacency Matrix** or an **Adjacency List**?

2. Priority Queue Implementation

How do we efficiently find the node with the smallest current distance?

- Should we use a simple **Array** or a binary **Min-Heap**?

We will compare two combinations:

- **Implementation A:** Adjacency Matrix + Array-based Queue
- **Implementation B:** Adjacency List + Min-Heap



Pseudocode: Implementation A

```
DIJKSTRA_AdjMat(adjMatrix, source):
```

```
    dist = [ $\infty$ ] * V
```

```
    visited = [False] * V
```

```
    prev = [-1] * V
```

```
    dist[source] = 0
```

```
    for each vertex to process:
```

```
        u = find unvisited node with min distance
```

```
    // linear scan
```

```
        mark u as visited
```

```
        for each potential neighbor v in graph:
```

```
            if u connected to v and v not
```

```
visited:
```

```
            if new path through u is shorter:
```

```
                update distance to v
```

```
                record u as predecessor of v
```

1. Linear Scan for Minimum

- Performs brute-force search through all nodes in every iteration
- Repeatedly scans entire node list

2. Exhaustive Neighbour Checks

- Adjacency matrix requires checking all V nodes to see if they're connected to current node u.

3. Early Termination

- Can stop early if no more reachable nodes exist.

4. Path Reconstruction

- Maintains prev array to track how we reached each node.
- Tracks the actual route to each node, not just distance.

5. Performance Characteristic

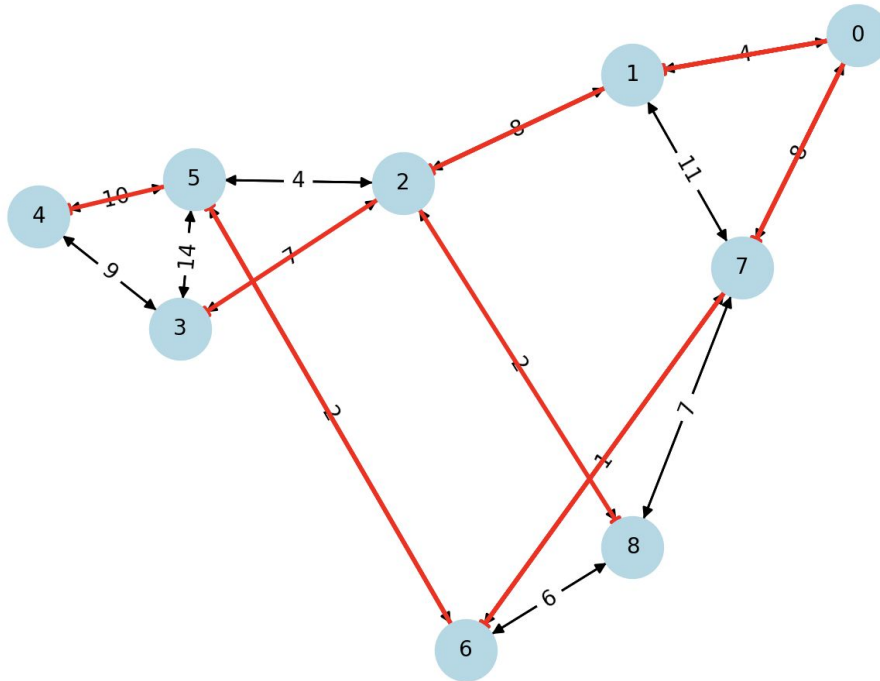
- Double the nodes = quadruple the work (quadratic scaling).

Algorithm (A) Implementation & Example Graph

```
1 def dijkstra_AdjMat(adjMatrix, source):
2     INF = sys.maxsize
3     V = len(adjMatrix)
4     dist = [INF] * V
5     visited = [False] * V
6     pi = [-1] * V
7
8     dist[source] = 0
9
10    for _ in range(V - 1):
11        u = -1
12        for j in range(V):
13            if not visited[j] and (u == -1 or dist[j] < dist[u]):
14                u = j
15
16        if u == -1:
17            break
18
19        visited[u] = True
20
21        for v in range(V):
22            if not visited[v] and adjMatrix[u][v] != INF:
23                if dist[u] + adjMatrix[u][v] < dist[v]:
24                    dist[v] = dist[u] + adjMatrix[u][v]
25                    pi[v] = u
26
27    # for k in range(V):
28    #     print(f"Distance to {k}: {dist[k]}")
29
30    return dist, pi
```

```
1 graph = [
2     [0, 4, INF, INF, INF, INF, INF, 8, INF],
3     [4, 0, 8, INF, INF, INF, INF, 11, INF],
4     [INF, 8, 0, 7, INF, 4, INF, INF, 2],
5     [INF, INF, 7, 0, 9, 14, INF, INF, INF],
6     [INF, INF, INF, 9, 0, 10, INF, INF, INF],
7     [INF, INF, 4, 14, 10, 0, 2, INF, INF],
8     [INF, INF, INF, INF, INF, 2, 0, 1, 6],
9     [8, 11, INF, INF, INF, INF, 1, 0, 7],
10    [INF, INF, 2, INF, INF, INF, 6, 7, 0]
11 ]
12 source = 0
```

Algorithm (A) Results



Red = Shortest Path

Black = Edges



Time Complexity Analysis - A

Graph Representation: Adjacency matrix $\rightarrow$ checking neighbors takes $O(V)$ time per vertex.

Priority Queue: Simple array $\rightarrow$ finding the minimum unvisited vertex takes $O(V)$ time.

Total Complexity:

- Outer loop runs V times.
- Each iteration does:
 - $O(V)$ to find min vertex.
 - $O(V)$ to scan neighbors.

Overall: $O(V^2)$ regardless of edge count.



Pseudocode: Implementation B

```
DIJKSTRA_MinHeap(adjList, source):
```

```
    dist = [ $\infty$ ] * V
```

```
    prev = [-1] * V
```

```
    dist[source] = 0
```

```
    heap = [(0, source)]
```

```
    while heap not empty:
```

```
        pop node u with min distance from heap
```

```
        if u has outdated distance: skip
```

```
        for each actual neighbor (v, weight) of u:
```

```
            if new path through u is shorter:
```

```
                update distance to v
```

```
                record u as predecessor of v
```

```
                push new distance to heap
```

1. Instant Minimum Node Access

- Min-heap provides direct access to closest node
- Eliminates need for costly linear scans

2. Direct Neighbour Processing

- Adjacency list only checks actual connections
- No time wasted on checking non-existing edges

3. Efficient Updates with Lazy Deletion

- Pushes new distance values into the heap
- Skips processing if popped distance is outdated

4. Performance Characteristic

- Much faster for sparse networks
- Performance scales efficiently with graph size
- Time complexity: $O((V + E) \log V)$

Algorithm (B) Implementation & Example Graph

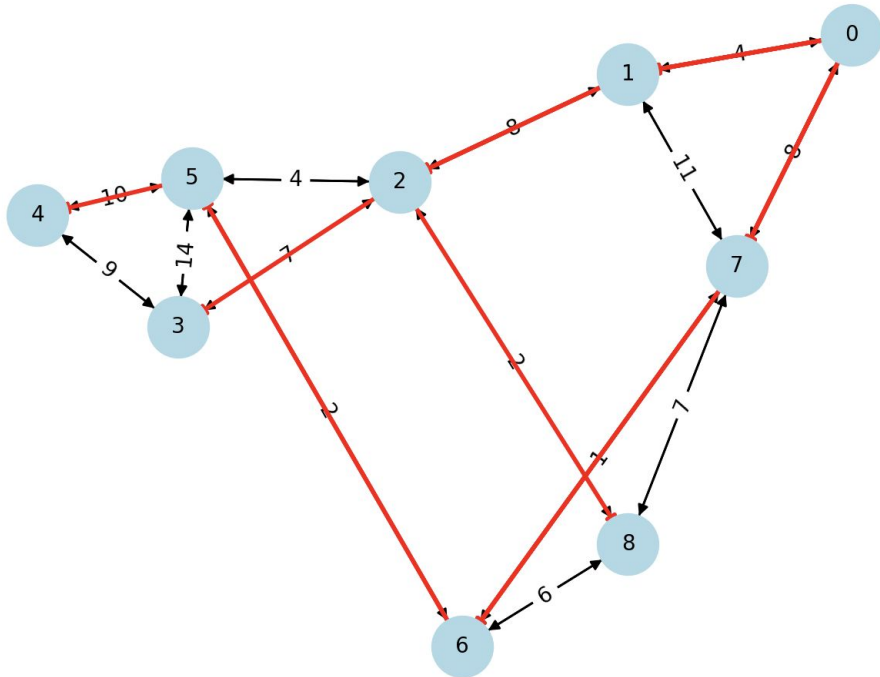


```
1 def dijkstra_MinHeap(adjList, source):
2     INF = sys.maxsize
3     V = len(adjList)
4     dist = [INF] * V
5     pi = [-1] * V
6     dist[source] = 0
7
8     # priority queue
9     pq = [(0, source)]
10
11     while pq:
12         d_u, u = heapq.heappop(pq)
13
14         if d_u > dist[u]:
15             continue
16
17         for v, weight in adjList[u]:
18             if dist[u] + weight < dist[v]:
19                 dist[v] = dist[u] + weight
20                 pi[v] = u
21                 heapq.heappush(pq, (dist[v], v))
22
23     # for i in range(V):
24     #     print(f"Distance to {i}: {dist[i]}")
25
26     return dist, pi
```



```
1 adj_list = [
2     [(1, 4), (7, 8)],
3     [(0, 4), (2, 8), (7, 11)],
4     [(1, 8), (3, 7), (5, 4), (8, 2)],
5     [(2, 7), (4, 9), (5, 14)],
6     [(3, 9), (5, 10)],
7     [(2, 4), (3, 14), (4, 10), (6, 2)],
8     [(5, 2), (7, 1), (8, 6)],
9     [(0, 8), (1, 11), (6, 1), (8, 7)],
10    [(2, 2), (6, 6), (7, 7)]
11 ]
12
13 source = 0
```

Algorithm (B) Results





Time Complexity Analysis - B

Graph Representation: Adjacency list $\rightarrow$ accessing neighbors is $O(1)$ per edge.

Priority Queue: Min-heap $\rightarrow$ insert/update is $O(\log V)$.

Total Complexity:

- Each vertex is inserted once $\rightarrow O(V \log V)$.
- Each edge may cause a decrease-key $\rightarrow O(E \log V)$.

Overall: $O((V + E) \log V)$ — much faster for sparse graphs.



Empirical Analysis

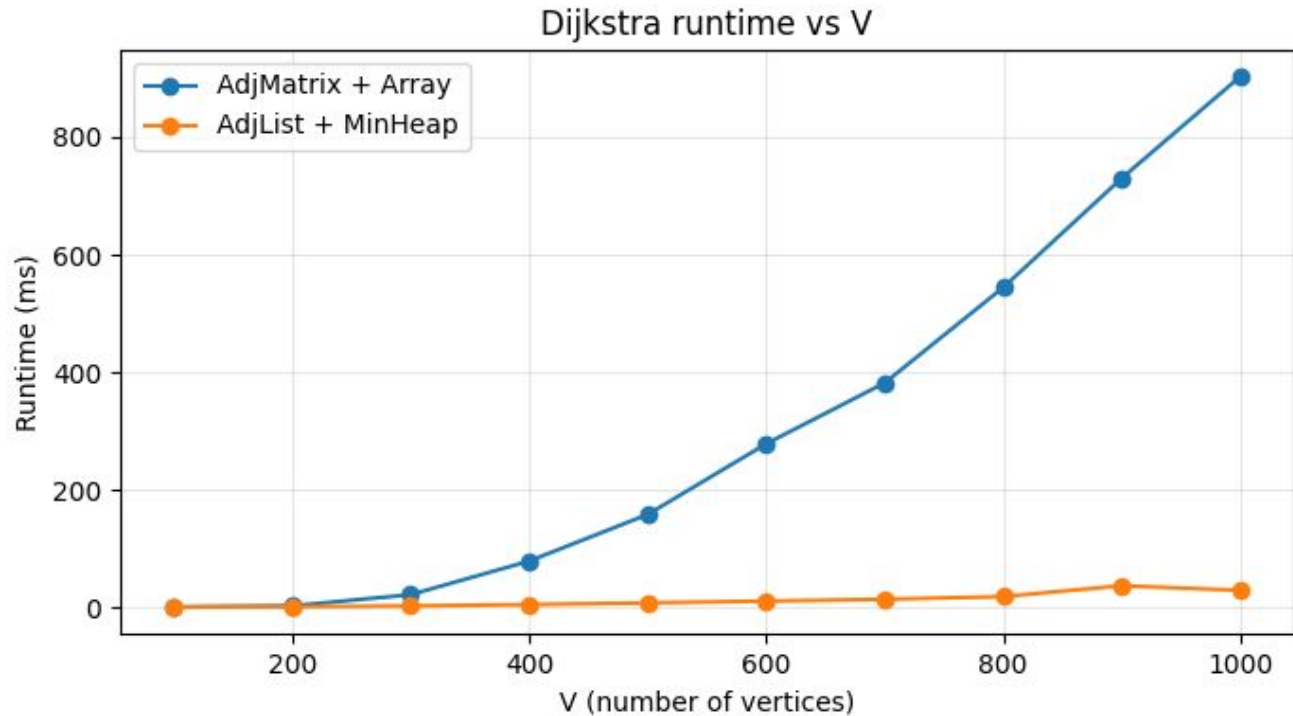
Vertices(V)	Adj Matrix Time(s)	Min Heap Time(s)
100	0.00089	0.00024
200	0.00341	0.00064
400	0.01314	0.00188
800	0.05972	0.00681



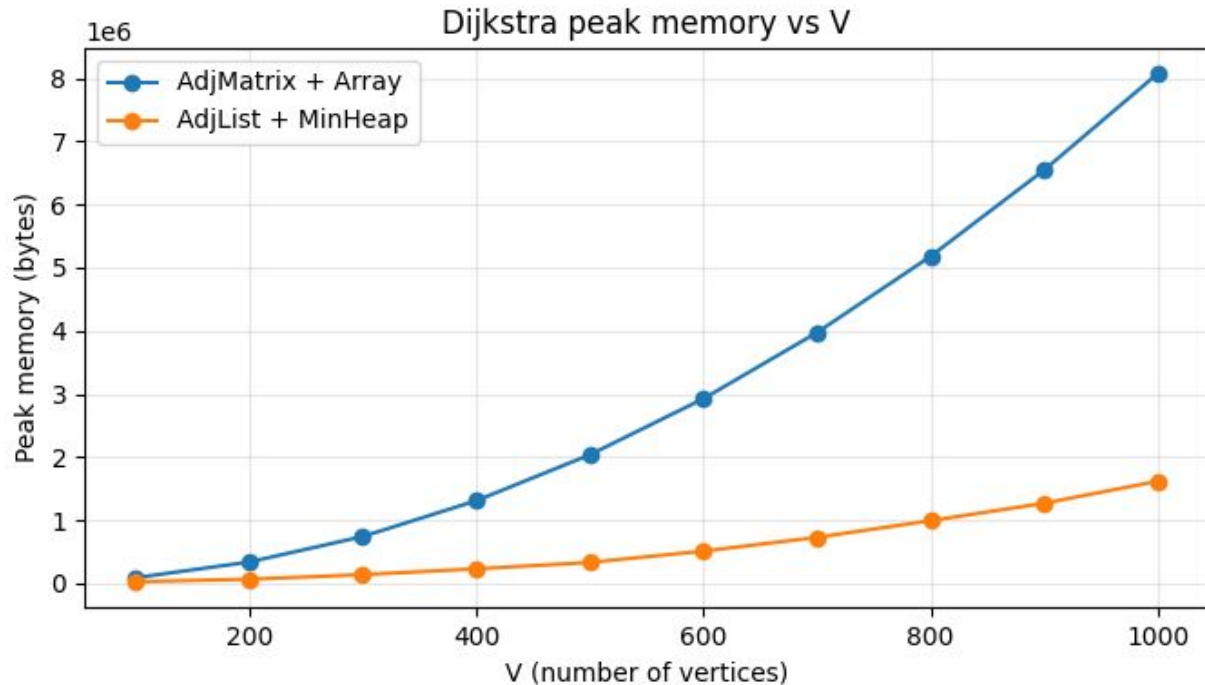
c) Comparing (a) and (b)

	Adjacency Matrix + Array-based Queue	Adjacency List + Min-Heap
Representation	Every vertex pair is represented regardless of whether an edge exists	Non-edges are omitted and not stored
Selection of next Vertex	Scans all unvisited vertices for minimum tentative distance each iteration	Delegates selection to heap operations instead of full scan in outer loop
Time complexity	$O(V^2)$	$O((V+E) \log V)$
Space complexity	$O(V^2)$	$O(V+E)$

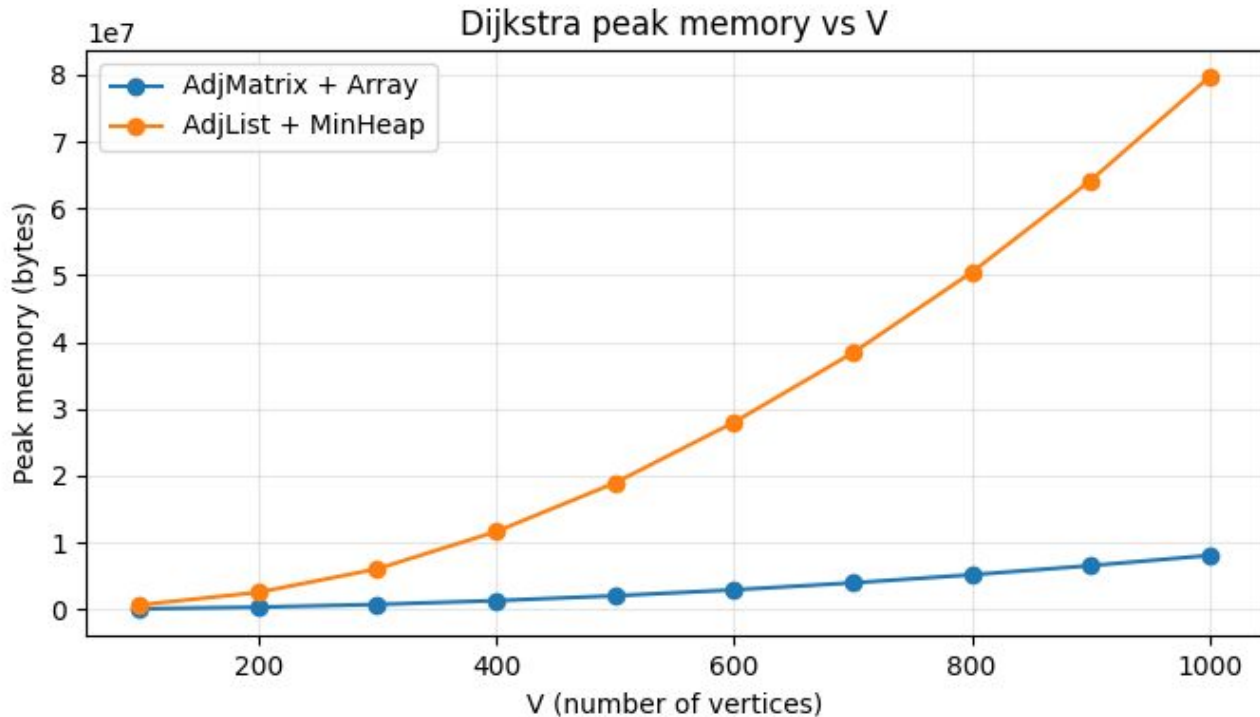
Runtime



Peak memory with low edge probability



Peak memory with high edge probability



c) Which implementation is better?

```
== Implementation A (Matrix + Array) ==  
Time: 0.014528s  
Peak memory: 744388 bytes  
Matrix memory: 739128 bytes
```

Time complexity : $O(V^2)$
Space complexity : $O(V^2)$

```
== Implementation B (List + Heap) ==  
Time: 0.000359s  
Peak memory: 168980 bytes  
List memory: 151904 bytes
```

Time complexity : $O((V+E) \log V)$
Space complexity : $O(V+E)$

- Part B runtime is significantly lesser than Part A
- Part B also require less memory compared to Part A

Is there an algorithm that is clearly better?

- Most of the time, adjacency list + min heap (Part B) will be better
- However, adjacency matrix + array (Part A) will be better if the graph is very dense or small



c) Generating input for (a and b)

```
def gen_random_adj_matrix(V, p, wmin=1, wmax=10,
seed=None):
    if seed is not None:
        random.seed(seed)
    M = [[INF]*V for _ in range(V)]
    for i in range(V):
        M[i][i] = 0
    for i in range(V):
        for j in range(i+1, V):
            if random.random() < p:
                w = random.randint(wmin, wmax)
                M[i][j] = w
                M[j][i] = w
    return M
```

```
def gen_random_adj_list(V, p, wmin=1, wmax=10,
seed=None):
    if seed is not None:
        random.seed(seed)
    adj = [[] for _ in range(V)]
    for u in range(V):
        for v in range(u+1, V):
            if random.random() < p:
                w = random.randint(wmin, wmax)
                adj[u].append((v, w))
                adj[v].append((u, w))
    return adj
```